

Python — La visualisation de données

Document 999 — Cheat sheets

Série « Visualisation » · référence transversale

Table des matières

1. Comment utiliser cette référence
 2. Les imports conventionnels
 3. Couleurs, marqueurs et styles de ligne
 4. Les colormaps
 5. rcParams et thèmes
 6. Patterns Matplotlib fréquents
 7. Patterns Pandas, Seaborn, Plotly
 8. Aide-mémoire : quelle bibliothèque ?
 9. Pièges fréquents
- Récapitulatif

1. Comment utiliser cette référence

Ce document ne s'apprend pas : il se consulte. Il rassemble, en tableaux, les éléments qu'on cherche sans cesse en traçant — un nom de colormap, un code de marqueur, un rcParam, le squelette d'un pattern. Les explications sont ailleurs (dans les documents 10 à 70) ; ici, on va droit au bon code.

Pourquoi ça compte. Garder ce document sous la main pendant qu'on code. Il clôt la série en répondant aux « c'était quoi déjà le code du marqueur triangle ? » sans rouvrir un cours entier.

2. Les imports conventionnels

Les alias standard de la série. Les respecter rend le code lisible par quiconque connaît l'écosystème (document 00).

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.ticker as mticker
import matplotlib.gridspec as gridspec
import matplotlib.patches as mpatches
import matplotlib.image as mpimg
from matplotlib.animation import FuncAnimation
from mpl_toolkits.mplot3d import Axes3D      # 3D (doc 60)

import pandas as pd                          # df.plot (doc 20)
import seaborn as sns                       # statistique (doc 30)
import plotly.express as px                 # interactif (doc 40)
import plotly.graph_objects as go
import pyqtgraph as pg                      # temps reel (doc 50)
from PIL import Image, ImageDraw, ImageFont# images (doc 70)
```

3. Couleurs, marqueurs et styles de ligne

3.1 Couleurs de base

Matplotlib comprend les noms, les codes courts, les hexadécimaux et les niveaux de gris.

Écriture	Exemple
Nom	"steelblue", "tomato", "forestgreen"
Code court	"b" (bleu), "r" (rouge), "g" (vert), "k" (noir)
Hexadécimal	"#1A4A6B", "#FF6600"
Niveau de gris	"0.0" (noir) à "1.0" (blanc), ex. "0.5"
Cycle par défaut	C0, C1, C2... (couleurs successives du thème)

3.2 Marqueurs

Code	Forme	Code	Forme
"o"	Cercle	"s"	Carré
"^"	Triangle haut	"v"	Triangle bas
"D"	Losange	"*"	Étoile
"+"	Plus	"x"	Croix
","	Point	"None"	Aucun

3.3 Styles de ligne

Code	Style	Équivalent long
"_"	Continue	"solid"
"_ _"	Tirets	"dashed"
". "	Pointillés	"dotted"
"_ ."	Tiret-point	"dashdot"

4. Les colormaps

Une colormap (cmap) mappe une valeur numérique sur une couleur. Le choix dépend de la nature de la donnée — c'est le réflexe le plus important de la section.

Type	Colormaps	Pour quelle donnée
Séquentielle	viridis, plasma, magma	Grandeur croissante (volume, intensité).
Divergente	coolwarm, RdBu, seismic	Écart de part et d'autre d'un centre (corrélacion, P&L).
Cyclique	twilight, hsv	Donnée circulaire (angle, heure, phase).
Qualitative	tab10, Set2, Paired	Catégories distinctes (actifs, secteurs).

```
# valeur continue -> séquentielle
ax.scatter(x, y, c=valeurs, cmap="viridis")

# matrice de corrélation -> divergente centrée sur 0
sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1)

# suffixe _r pour inverser une colormap
ax.imshow(img, cmap="gray_r")
```

Pourquoi ça compte. Préférer viridis (séquentielle) à l'ancien jet : viridis est lisible même en noir et blanc et pour les daltoniens. Pour une corrélation ou un P&L, une divergente centrée sur 0 (coolwarm) ; jamais une séquentielle, qui masquerait le signe.

5. rcParams et thèmes

rcParams fixe les valeurs par défaut globales, une fois pour toute la session (document 10). Les plus utiles :

rcParam	Effet
<code>plt.rcParams["figure.figsize"]</code>	Taille par défaut des figures (pouces).
<code>plt.rcParams["figure.dpi"]</code>	Résolution d'affichage.
<code>plt.rcParams["savefig.dpi"]</code>	Résolution à l'export.
<code>plt.rcParams["font.size"]</code>	Taille de police de base.
<code>plt.rcParams["axes.grid"]</code>	Grille activée par défaut (True/False).
<code>plt.rcParams["lines.linewidth"]</code>	Épaisseur de ligne par défaut.

```
# régler plusieurs defaults d'un coup
plt.rcParams.update({
    "figure.figsize": (8, 4),
    "figure.dpi": 120,
    "axes.grid": True,
    "font.size": 11,
})

# ou un theme tout fait
plt.style.use("ggplot")           # Matplotlib
sns.set_theme(style="whitegrid")  # Seaborn
```

Thèmes utiles. Côté Matplotlib : ggplot, seaborn-v0_8, bmh, fivethirtyeight. Côté Seaborn : les styles whitegrid, darkgrid, white, ticks. `plt.style.available` liste tout ce qui est installé.

6. Patterns Matplotlib fréquents

Le squelette universel. Le point de départ de presque tout (document 10).

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, label="serie")
ax.set_title("..."); ax.set_xlabel("..."); ax.set_ylabel("...")
ax.legend(); ax.grid(True, alpha=0.3)
```

```
fig.tight_layout()
fig.savefig("sortie.png", dpi=150)
```

Axe de dates. Formater un axe temporel (séries de prix) avec matplotlib.dates.

```
import matplotlib.dates as mdates
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
ax.xaxis.set_major_locator(mdates.MonthLocator())
fig.autofmt_xdate() # incline les dates
```

Axe en pourcentage ou en milliers. Formater les graduations avec matplotlib.ticker.

```
import matplotlib.ticker as mticker
ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1))
ax.yaxis.set_major_formatter(mticker.StrMethodFormatter("{x:,.0f}"))
```

Second axe y. Superposer deux échelles (prix et volume).

```
ax2 = ax.twinx() # partage l'axe x
ax.plot(jours, prix); ax2.bar(jours, volume, alpha=0.3)
```

7. Patterns Pandas, Seaborn, Plotly

Pandas (doc 20). Explorer vite, puis récupérer l'Axes.

```
ax = df.plot(y=["ES", "NQ"], title="...", figsize=(8, 4), grid=True)
df["rendement"].plot.hist(bins=30)
fig = ax.get_figure(); fig.savefig("sortie.png")
```

Seaborn (doc 30). Statistique, données tidy, axes-level vs figure-level.

```
sns.set_theme(style="whitegrid")
ax = sns.scatterplot(data=df, x="es", y="nq", hue="regime") # axes-level
g = sns.relplot(data=df, x="es", y="nq", col="regime") # figure-level
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", vmin=-1, vmax=1)
```

Plotly (doc 40). Interactif, sortie HTML.

```
fig = px.line(df, x="date", y="prix", color="actif", title="...")
fig.update_layout(hovermode="x unified", template="plotly_white")
fig.write_html("rapport.html") # interactif
fig.write_image("rapport.png") # fige (kaleido)
```

8. Aide-mémoire : quelle bibliothèque ?

Le tableau de décision de toute la série, condensé. La question reste : quelle est la sortie voulue (document 00) ?

Besoin	Outil	Sortie
Rapport, PDF figé	Matplotlib	Image (PNG/PDF/SVG)
Coup d'œil rapide	Pandas .plot	Image (via Matplotlib)
Statistique, style	Seaborn	Image (via Matplotlib)
Web, dashboard	Plotly	HTML interactif
Flux temps réel	PyQtGraph	Fenêtre Qt (GPU)
Surface, 3D figée	Matplotlib 3D	Image 3D projetée
Images, overlays	Matplotlib / Pillow	Image

9. Pièges fréquents

Piège	Le réflexe correct
<code>legend()</code> n'affiche rien	Passer <code>label=</code> à chaque tracé d'abord.
Axes vs Axis	Axes = la zone de tracé ; Axis = un seul axe (x ou y).
<code>plt.show()</code> vide la figure	Appeler <code>savefig</code> AVANT <code>show()</code> .
scatter sans <code>x=/y=</code> (Pandas)	Nommer explicitement <code>x=</code> et <code>y=</code> .
Surface sans meshgrid	Construire X, Y avec <code>np.meshgrid</code> d'abord.
jet pour une corrélation	Une divergente centrée (<code>coolwarm</code> , <code>vmin/vmax</code>).
Recréer la courbe en temps réel	<code>setData</code> sur l'objet existant (<code>PyQtGraph</code>).
Mémoire qui gonfle en boucle	<code>plt.close(fig)</code> après chaque figure.

Pourquoi ça compte. La moitié des bugs de visualisation tiennent à cette liste. La relire avant de déboguer fait souvent gagner l'heure qu'on s'apprêtait à perdre.

Récapitulatif

- 1. Une référence, pas un cours.** Ce document se consulte ; les explications sont dans les documents 10 à 70.
- 2. Les imports.** Respecter les alias conventionnels (`plt`, `sns`, `px`, `pg...`) rend le code lisible.
- 3. Couleurs et marqueurs.** Noms, codes courts, hex ; marqueurs (`o`, `s`, `^`) et styles (`-`, `--`, `:`).
- 4. Colormaps.** Séquentielle (`viridis`) pour une grandeur, divergente (`coolwarm`) pour un écart signé, qualitative pour des catégories.
- 5. rcParams et thèmes.** Régler les défauts globaux ; `plt.style.use` / `sns.set_theme` pour un thème complet.
- 6–7. Patterns.** Squelette `fig`, `ax` ; axes de dates/pourcentages ; `df.plot` ; axes-level vs figure-level ; `px` + `write_html`.
- 8. Le choix.** La sortie voulue décide de l'outil — le fil conducteur de toute la série.
- 9. Les pièges.** `label` avant `legend`, `savefig` avant `show`, `meshgrid` avant surface, `setData` en temps réel.

Fin de la série « Visualisation ». Des fondations statiques (*Matplotlib*, *Pandas*, *Seaborn*) à l'interactif (*Plotly*), au temps réel (*PyQtGraph*), à la 3D et aux images : sept bibliothèques, un seul réflexe — choisir selon la sortie voulue.